UNIVERSIDAD DEL PACÍFICO

Solucionario para el examen final

Aproximación numérica y optimización

Las soluciones siguen la numeración de la lista de 19 ejercicios. Cada bloque de código se puede ejecutar por separado con Python y las bibliotecas que importa.

Aproximación por mínimos cuadrados

1. Ecuaciones normales y geometría del ajuste

a) Para $r = b - A\widehat{\theta}$ y cualquier $v \in \mathbb{R}^n$,

$$\|b - A(\widehat{\theta} + tv)\|_2^2 = \|r\|_2^2 - 2t\, v^T A^T r + t^2 \|Av\|_2^2.$$

Si $\widehat{\theta}$ minimiza el error, la derivada respecto de t en $t = 0$ se anula para todo v. Así, $v^T A^T r = 0$ para todo v y $A^T r = 0$. Recíprocamente, si $A^T r = 0$, el término cruzado se anula. Al tomar $t = 1$ y $v = \theta - \widehat{\theta}$ se obtiene

$$\|b - A\theta\|_2^2 = \|b - A\widehat{\theta}\|_2^2 + \|A(\theta - \widehat{\theta})\|_2^2.$$

El último término es no negativo, lo cual prueba suficiencia.

b) Existe un minimizador porque la proyección ortogonal de b sobre $\operatorname{im} A$ existe y pertenece a $\operatorname{im} A$. La identidad anterior muestra que la igualdad de errores ocurre exactamente si $\theta - \widehat{\theta} \in \ker A$. El conjunto de minimizadores es $\widehat{\theta} + \ker A$. Si A tiene rango columna completo, $\ker A = \{0\}$ y el coeficiente es único. En ambos casos todos los minimizadores producen el mismo vector $A\widehat{\theta}$.

2. Ajuste de costos mediante QR

a) Sea $\theta = (a, b, c)^T$, $A_{i,:} = (1, q_i, q_i^2)$ y $\mathbf{C} = (4, 7, 11, 19, 28)^T$. Se minimiza $\|\mathbf{C} - A\theta\|_2^2$. Si $A\theta = 0$, un polinomio de grado a lo sumo dos tiene cinco raíces distintas, así que todos sus coeficientes son cero y A tiene rango tres.

b) Con $A = QR$ y $Q^T Q = I$, se tiene la descomposición ortogonal

$$\|\mathbf{C} - QR\theta\|_2^2 = \|Q^T\mathbf{C} - R\theta\|_2^2 + \|(I - QQ^T)\mathbf{C}\|_2^2.$$

Como R es invertible, la primera norma se anula al resolver $R\widehat{\theta} = Q^T\mathbf{C}$ por sustitución hacia atrás.

2. Ajuste de costos mediante QR (continuación)

```
import numpy as np
from scipy.linalg import solve_triangular

q = np.arange(5, dtype=float)
costos = np.array([4, 7, 11, 19, 28], dtype=float)
A = np.column_stack((np.ones_like(q), q, q**2))
Q, R = np.linalg.qr(A, mode="reduced")
a, b, c = coef = solve_triangular(R, Q.T @ costos)
r = costos - A @ coef
print(coef)
print(np.linalg.norm(r), np.linalg.norm(A.T @ r))
print(10 * (b + 2 * c * 2.5))
```

El cálculo da $(\widehat{a}, \widehat{b}, \widehat{c}) \approx (4.08571429, 1.42857143, 1.14285714)$. Para $r = \mathbf{C} - A\widehat{\theta}$, $\|r\|_2 \approx 0.71713717$ mide la magnitud del desajuste y $\|A^T r\|_2 \approx 5.35 \times 10^{-14}$ verifica la ortogonalidad del residuo dentro de la precisión del cálculo.

c) La derivada estimada en $q = 2.5$ es $\widehat{b} + 2\widehat{c}(2.5) \approx 7.14285714$ miles de soles por cien unidades. El costo marginal es **71.43 soles por unidad**.

3. Proyección en una base ortogonal

a) La condición $\langle 1, x^2 - \mu \rangle = 2/3 - 2\mu = 0$ da $\mu = 1/3$. Por paridad x es ortogonal a los otros dos polinomios. Los tres son no nulos y tienen grados distintos, por lo que forman una base de $\mathcal{P}_2$.

b) Sea $\phi = x^2 - 1/3$. El coeficiente de x en la base ortogonal es cero por paridad de x^5. Para los términos pares,

$$\frac{\langle x^4, 1 \rangle}{\langle 1, 1 \rangle} = \frac{1}{5}, \qquad \langle x^4, \phi \rangle = \frac{16}{105}, \qquad \langle \phi, \phi \rangle = \frac{8}{45}.$$

La proyección es $p_*(x) = \frac{1}{5} + \frac{6}{7}(x^2 - \frac{1}{3}) = \frac{6}{7}x^2 - \frac{3}{35}$. El residuo es ortogonal a toda $\mathcal{P}_2$, así que para cualquier $p \in \mathcal{P}_2$ el error cuadrático de p es el de p_* más $\int_{-1}^{1} (p - p_*)^2 \, dx$.

c) Por ortogonalidad,

$$\int_{-1}^{1} (x^4 - p_*)^2 \, dx = \frac{2}{9} - \frac{2}{25} - \frac{6}{7}\frac{16}{105} = \frac{128}{11025} \approx 0.011610.$$

El interpolante en $-1, 0, 1$ es $p_I(x) = x^2$. Su error es $\frac{2}{9} - \frac{4}{7} + \frac{2}{5} = \frac{16}{315} \approx 0.050794$, es decir, 35/8 veces el del ajuste por proyección.

Interpolación

4. Diferencias divididas y cota del error

a) La tabla muestra también la fila que se agrega en el inciso b).

x_i	$f[x_i]$	Primer orden	Segundo orden	Tercer orden
0	1			
1	2	1		
3	10	4	1	
4	19	9	5/3	1/6

Por ejemplo, $f[0,1,3] = (4-1)/(3-0) = 1$. Luego $P_2(x) = 1 + x + x(x-1) = 1 + x^2$ y $P_2(2) = 5$.

b) El nuevo coeficiente es $f[0,1,3,4] = (5/3-1)/4 = 1/6$. Por tanto, $P_3(x) = 1+x^2+x(x-1)(x-3)/6$ y $P_3(2) = 14/3$. La aproximación disminuye en $1/3$.

c) Sea $\omega(x) = x(x-1)(x-3)(x-4)$. Para ver la fórmula del resto en $x = 2$, defina $c_* = (f(2) - P_3(2))/\omega(2)$. La función $f - P_3 - c_*\omega$ se anula en los cuatro nodos y en 2. Cuatro aplicaciones de Rolle dan un $\xi \in (0,4)$ con $f^{(4)}(\xi) - 24c_* = 0$. Como $\omega(2) = 4$, el error es a lo sumo $(6/24)4 = 1$. Las dos funciones $f_\pm = P_3 \pm \omega/4$ interpolan los cuatro datos, cumplen $|f_\pm^{(4)}| = 6$ y toman los valores $14/3 \pm 1$ en 2. La cota se alcanza.

5. Hermite y datos de pendiente

a) Los datos son $f(0) = 0$, $f'(0) = 1$, $f(1) = \log 2$ y $f'(1) = 1/2$. Escriba $H(x) = x + ax^2 + bx^3$. Las condiciones en 1 dan $a + b = \log 2 - 1$ y $2a + 3b = -1/2$. Luego

$$H(x) = x + (3\log 2 - 5/2)x^2 + (3/2 - 2\log 2)x^3.$$

La diferencia de dos soluciones tendría raíces dobles en 0 y 1, por lo que sería divisible por $x^2(x-1)^2$. Al tener grado a lo sumo tres, la diferencia debe ser idénticamente cero.

b) Como $f^{(4)}(x) = -6/(1+x)^4$, el resto de Hermite satisface $|f(1/2) - H(1/2)| \le (6/24)(1/2)^4 = 1/64$. La evaluación directa da

$$H(1/2) = \tfrac{1}{2}\log 2 + \tfrac{1}{16}, \qquad f(1/2) - H(1/2) \approx -0.00360848.$$

El interpolante lineal es $L(x) = x\log 2$ y su error firmado en $1/2$ es $\log(3/2) - \tfrac{1}{2}\log 2 \approx 0.05889152$. En valor absoluto, el error de Hermite es aproximadamente 0.00360848.

Interpolación (continuación)

6. Spline natural y factibilidad

a) Por las condiciones naturales, $M_0 = M_2 = 0$. En el primer tramo, la segunda derivada lineal vale $S''(m) = M_1 m$. Tras integrar y usar $S(0) = 0$, $S(1) = 1$,

$$S(m) = m + \frac{M_1}{6}(m^3 - m), \qquad S'(1^-) = 1 + \frac{M_1}{3}.$$

En el segundo tramo, con $t = m - 1$, se tiene $S''(m) = M_1(1 - t)$. Integrar y usar $S(1) = 1$ y $S(2) = 8/5$ da

$$S(m) = 1 + \left(\frac{3}{5} - \frac{M_1}{3}\right) t + \frac{M_1}{2} t^2 - \frac{M_1}{6} t^3.$$

Al igualar las derivadas en $m = 1$ resulta $M_1 = -3/5$. En $[0, 1]$, $S(m) = \frac{11}{10} m - \frac{1}{10} m^3$ y $S(1/2) = 43/80 = 0.5375$.

b) Para $0 < m < 1$, $S(m) - m = m(1 - m^2)/10 > 0$, por lo que el spline viola $c \le m$. Para dos nodos factibles (m_a, c_a) y (m_b, c_b), escriba $m = (1 - t)m_a + t m_b$ con $0 \le t \le 1$. Entonces

$$0 \le L(m) = (1 - t)c_a + t c_b \le (1 - t)m_a + t m_b = m,$$

pues ambos pesos son no negativos.

7. Fenómeno de Runge y nodos de Chebyshev

a)–b) Se usan $n + 1$ nodos equiespaciados y los ceros $x_j = \cos\left((2j + 1)\pi/[2(n + 1)]\right)$, $j = 0, \ldots, n$, de T_{n+1}. La tabla usa la misma malla de 2001 puntos en ambos casos.

n	Error máx. equiespaciado	Error máx. Chebyshev	$\lvert x \rvert$ en el máximo equiespaciado
8	1.045174	0.170834	0.920
16	14.393851	0.032613	0.967
24	257.212912	0.006948	0.980

Los picos de los nodos equiespaciados aparecen entre nodos cercanos a los extremos. En los extremos mismos el error equiespaciado se anula, pues -1 y 1 son nodos de ese interpolante.

c) El factor que depende explícitamente de los nodos en el resto $f^{(n+1)}(\xi_x)\omega(x)/(n + 1)!$ es $\omega(x) = \prod_{j=0}^{n}(x - x_j)$. Con los ceros de T_{n+1}, $\omega(x) = 2^{-n} T_{n+1}(x)$ porque T_{n+1} tiene coeficiente principal 2^n. Así, $\max_{[-1,1]} |\omega(x)| = 2^{-n}$.

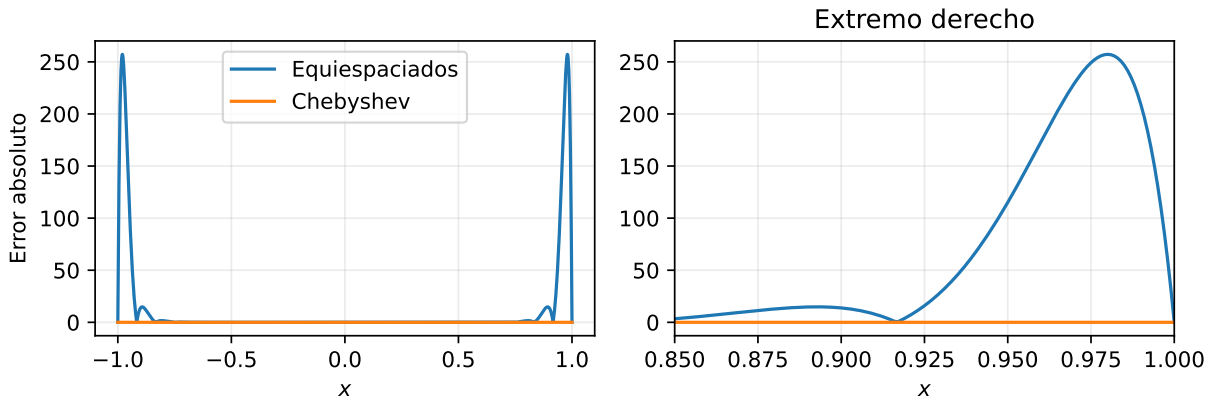
7. Código y gráfica del error para grado 24

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import BarycentricInterpolator

def f(x):
    return 1 / (1 + 25 * x**2)

malla = np.linspace(-1, 1, 2001)
for n in (8, 16, 24):
    j = np.arange(n + 1)
    nodos_e = np.linspace(-1, 1, n + 1)
    nodos_c = np.cos((2 * j + 1) * np.pi / (2 * (n + 1)))
    errores = []
    for nodos in (nodos_e, nodos_c):
        p = BarycentricInterpolator(nodos, f(nodos))
        errores.append(np.abs(f(malla) - p(malla)))
    err_e, err_c = errores
    print(n, err_e.max(), err_c.max(),
          abs(malla[np.argmax(err_e)]))

fig, (ax, zoom) = plt.subplots(1, 2, figsize=(8, 2.85))
for panel in (ax, zoom):
    panel.plot(malla, err_e, label="Equiespaciados")
    panel.plot(malla, err_c, label="Chebyshev")
    panel.set_xlabel("$x$")
    panel.grid(alpha=0.25)
ax.set_ylabel("Error absoluto")
ax.legend()
zoom.set_xlim(0.85, 1.0)
zoom.set_title("Extremo derecho")
fig.tight_layout()
plt.show()
```



Integración numérica y cuadratura

8. Valor presente con trapecio y Simpson

a) El integrando descontado es $g(t) = 120e^{-(0.08+0.04)t}$. Por integración directa,

$$V = \int_0^6 g(t)\,dt = 1000(1 - e^{-0.72}) \approx 513.247744040.$$

b) Sea $h = 6/n$ y $g_i = g(ih)$. En la malla de $n+1$ nodos,

$$T_n = h\left(\frac{g_0 + g_n}{2} + \sum_{i=1}^{n-1} g_i\right),$$

$$S_n = \frac{h}{3}\left(g_0 + g_n + 4\sum_{\substack{1\le i<n\\ i\ \text{impar}}} g_i + 2\sum_{\substack{2\le i<n\\ i\ \text{par}}} g_i\right).$$

Regla	n	Valor presente	Error absoluto	Evaluaciones
Trapecio	6	513.863493568	0.615749528	7
Trapecio	12	513.401709126	0.153965086	13
Simpson	6	513.248334289	5.902493×10^{-4}	7
Simpson	12	513.247780978	3.693801×10^{-5}	13

Los cocientes E_6/E_{12} son 3.99928 y 15.97946, respectivamente, cercanos a 2^2 y 2^4.

c) $\max|g^{(4)}| = 120(0.12)^4$ porque $e^{-0.12t}$ decrece. La cota compuesta es

$$\frac{6h^4}{180}\max|g^{(4)}| = 0.00082944(6/n)^4.$$

Para $n = 4$ da 0.00419904 y para $n = 6$ da 0.00082944. Como decrece en n, el menor par admisible es 6.

9. De los polinomios de Laguerre a la cuadratura de Gauss

a) La recurrencia con $n = 0$ da $L_1 = 1 - x$ y con $n = 1$ da $2L_2 = (3 - x)(1 - x) - 1$. Por tanto,

$$L_0 = 1, \qquad L_1 = 1 - x, \qquad L_2 = 1 - 2x + \tfrac{1}{2}x^2.$$

Como $I(x^k) = k!$, $I(L_2) = 1 - 2 + 2!/2 = 0$ e $I(xL_2) = 1 - 2(2!) + 3!/2 = 0$.

b) De $x^2 - 4x + 2 = 0$ se obtienen los nodos $x_1 = 2 - \sqrt{2}$ y $x_2 = 2 + \sqrt{2}$. La exactitud para 1 y x exige $w_1 + w_2 = 1$ y $w_1x_1 + w_2x_2 = 1$. Así,

$$w_1 = \frac{x_2 - 1}{x_2 - x_1} = \frac{2 + \sqrt{2}}{4}, \qquad w_2 = \frac{1 - x_1}{x_2 - x_1} = \frac{2 - \sqrt{2}}{4}.$$

c) Si $\deg p \le 3$, la división da $p = qL_2 + r$ con $\deg q \le 1$ y $\deg r \le 1$. Como q es combinación de 1 y x, el inciso a) da $I(qL_2) = 0$. Además, $Q(qL_2) = 0$ porque los nodos son ceros de L_2. Para r hay exactitud por el inciso b), así que $I(p) = Q(p)$. En grado cuatro, $Q(L_2^2) = 0$ mientras que $I(L_2^2) > 0$, pues L_2^2 es positivo salvo en sus raíces y el peso es positivo.

d) La integral exacta es $I(g) = \int_0^\infty e^{-2x}\,dx = 1/2$. Con las dos evaluaciones $g(x_i) = e^{-x_i}$,

$$Q(g) = \frac{2 + \sqrt{2}}{4}e^{-(2-\sqrt{2})} + \frac{2 - \sqrt{2}}{4}e^{-(2+\sqrt{2})} \approx 0.479964204.$$

El error absoluto es $|I(g) - Q(g)| \approx 0.020035796$.

Integración numérica y cuadratura (continuación)

10. Gauss–Hermite para una esperanza

a) Como $\sqrt{C} = e^{0.10+0.175Z}$, el cambio $z = \sqrt{2}\,x$ en la integral normal da

$$U = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} e^{0.10+0.175\sqrt{2}\,x} e^{-x^2}\, dx.$$

El factor exterior es $1/\sqrt{\pi}$ y se evalúa $g(x) = e^{0.10+0.175\sqrt{2}\,x}$ en los nodos.

b) La función generadora de momentos da $U = e^{0.10+(0.175)^2/2} \approx 1.1222240778058625$. Con los nodos x_i y pesos w_i de Hermite se calcula $U_n = \sum_i w_i g(x_i)/\sqrt{\pi}$.

```
import numpy as np
from numpy.polynomial.hermite import hermgauss

exacto = np.exp(0.10 + 0.175**2 / 2)
for n in (2, 4, 8):
    x, w = hermgauss(n)
    estimacion = (w @ np.exp(0.10 + 0.175 * np.sqrt(2) * x)
                  / np.sqrt(np.pi))
    print(n, f"{estimacion:.12f}",
          f"{abs(estimacion - exacto):.3e}")
```

n	U_n	$\lvert U_n - U\rvert$
2	1.122137080598	8.700×10^{-5}
4	1.122224077223	5.828×10^{-10}
8	1.122224077806	2.220×10^{-16}

El último error calculado está a escala de redondeo en doble precisión.

c) $C > 1$ si y solo si $Z > -0.20/0.35 = -4/7$, de modo que $\mathbb{P}(C > 1) = \Phi(4/7) \approx 0.716145417$. Tras el mismo cambio de variable, en Gauss–Hermite se evalúa $g_P(x) = \mathbf{1}_{\{x>-4/(7\sqrt{2})\}}$ y se mantiene el factor $1/\sqrt{\pi}$. La función tiene un salto en el umbral. La contribución de cada nodo cambia de cero a su peso completo al cruzar ese umbral. La regla conserva su exactitud para polinomios, pero los argumentos de error que requieren derivadas del integrando no se aplican a esta indicadora. Por eso el error puede variar de manera irregular al cambiar n.

Ecuaciones diferenciales

11. Acumulación de capital y ecuación integral

a) Integrar la EDO entre 0 y t da $k(t) = 10 + \int_0^t (6 - 0.3k(s))\,ds$. En un paso de longitud h, Euler usa el valor izquierdo del integrando, mientras que el trapecio usa el promedio de los dos extremos. Así,

$$k_{j+1}^{\mathrm{E}} = (1 - 0.3h)k_j^{\mathrm{E}} + 6h,$$

$$k_{j+1}^{\mathrm{T}} = k_j^{\mathrm{T}} + \tfrac{h}{2}\left[(6 - 0.3k_j^{\mathrm{T}}) + (6 - 0.3k_{j+1}^{\mathrm{T}})\right] = \frac{(1 - 0.15h)k_j^{\mathrm{T}} + 6h}{1 + 0.15h}.$$

b) Con $h = 1$, Euler da $k_1 = 13$ y $k_2 = 15.1$. El trapecio satisface $k_{j+1} - 20 = \frac{17}{23}(k_j - 20)$, de donde $k_1 = 290/23$ y $k_2 = 20 - 10(17/23)^2 = 7690/529$. El valor exacto $k(2) = 20 - 10e^{-0.6} \approx 14.51188364$ da

Método	k_2	Error absoluto
Euler explícito	15.10000000	0.58811636
Trapecio implícito	14.53686200	0.02497836

12. Condiciones de orden para RK2

a) Evalúe f, f_t, f_y en (t_j, y_j) y parta de $y_j = y(t_j)$. Por Taylor en las dos variables, $K_2 = f + ch(f_t + f_y f) + O(h^2)$. Sustituyendo en la fórmula de un paso,

$$y_{j+1} = y_j + hf + c\beta h^2(f_t + f_y f) + O(h^3).$$

Por la regla de la cadena, $y''(t_j) = f_t + f_y f$ y

$$y(t_j + h) = y_j + hf + \tfrac{1}{2}h^2(f_t + f_y f) + O(h^3).$$

Si $c\beta = 1/2$, la diferencia es $O(h^3)$. Para la necesidad basta tomar $f(t, y) = t$, que tiene $f_t + f_y f = 1$. En ese caso un valor $c\beta \neq 1/2$ deja un término de orden h^2 distinto de cero.

b) Con $h = 1/2$ e $y_0 = 1$, $K_1 = -1$. En Heun $(c, \beta) = (1, 1/2)$ y $K_2 = f(1/2, 1/2) = 0$, así que $y_1 = 1 + \frac{1}{2}(-1/2 + 0) = 3/4$. Para punto medio explícito $(c, \beta) = (1/2, 1)$ y $K_2 = f(1/4, 3/4) = -1/2$, así que $y_1 = 1 + \frac{1}{2}(-1/2) = 3/4$. Aquí ambos dan el mismo valor porque K_2 depende linealmente del parámetro c para esta f.

Ecuaciones diferenciales (continuación)

13. Orden observado y estabilidad

a) Cada método usa 1, 2 o 4 evaluaciones de f por paso, respectivamente. El código ejecuta las etapas de cada uno y compara el valor final con e^{-2}.

```
from math import exp, log2

def f(y):
    return -2 * y

def euler(y, h):
    return y + h * f(y)

def heun(y, h):
    k1 = f(y)
    k2 = f(y + h * k1)
    return y + h * (k1 + k2) / 2

def rk4(y, h):
    k1 = f(y)
    k2 = f(y + h * k1 / 2)
    k3 = f(y + h * k2 / 2)
    k4 = f(y + h * k3)
    return y + h * (k1 + 2*k2 + 2*k3 + k4) / 6

for metodo, etapas in ((euler, 1), (heun, 2), (rk4, 4)):
    anterior = None
    for n in (10, 20, 40, 80):
        y = 1.0
        for _ in range(n):
            y = metodo(y, 1 / n)
        error = abs(y - exp(-2))
        orden = "-" if anterior is None else log2(anterior / error)
        print(metodo.__name__, n, error, etapas * n, orden)
        anterior = error
```

13. Errores, órdenes observados y estabilidad

En cada fila con $N \geq 20$, la última columna corresponde al par $(N/2, N)$ y muestra $\log_2(e_{N/2}/e_N)$.

Método	N	Error absoluto	Evaluaciones de f	Orden observado
Euler	10	2.796110×10^{-2}	10	—
	20	1.375863×10^{-2}	20	1.0231
	40	6.823127×10^{-3}	40	1.0118
	80	3.397478×10^{-3}	80	1.0060
Heun	10	2.112748×10^{-3}	20	—
	20	4.871743×10^{-4}	40	2.1166
	40	1.171438×10^{-4}	80	2.0562
	80	2.873184×10^{-5}	160	2.0276
RK4	10	4.265194×10^{-6}	40	—
	20	2.451852×10^{-7}	80	4.1207
	40	1.469759×10^{-8}	160	4.0602
	80	8.996432×10^{-10}	320	4.0301

b) Para Euler, $y_{j+1} = (1-2h)y_j$. El decaimiento exige $|1-2h| < 1$, equivalente a $0 < h < 1$. Con $h = 1$ el factor es -1 y los valores alternan entre 1 y -1.

En Heun, el predictor es $(1-2h)y_j$. Al sustituirlo en el corrector se obtiene $y_{j+1} = (1-2h+2h^2)y_j$. Este factor es $2(h-1/2)^2 + 1/2 > 0$. Es menor que 1 si y solo si $2h(h-1) < 0$, es decir, $0 < h < 1$. En $h = 1$ el factor es 1 y todos los valores son 1.

Optimización sin restricciones

14. Gradiente con paso fijo y Newton

a) Con $D = \mathrm{diag}(1, 16)$, $\nabla f(x) = Dx$ y $x^{(k+1)} = (I - \alpha D)x^{(k)}$. Convergencia al minimizador desde cualquier punto exige $|1-\alpha| < 1$ y $|1-16\alpha| < 1$. La intersección es $0 < \alpha < 1/8$.

b) En ese intervalo, $1-\alpha > 0$. Para $0 < \alpha \leq 1/16$, domina $1-\alpha$. A partir de $1/16$, el máximo es el de las rectas $1-\alpha$ y $16\alpha - 1$. Su mínimo ocurre al igualarlas, en $\alpha_* = 2/17$. Fuera de $(0, 1/8)$ el máximo es al menos 1, así que el mínimo encontrado es global. Los factores de las componentes son $15/17$ y $-15/17$, de modo que

$$x^{(1)} = (15/17, -15/17)^T, qquad x^{(2)} = ((15/17)^2, (15/17)^2)^T.$$

La contracción es $15/17 = (\kappa_2(D) - 1)/(\kappa_2(D) + 1)$, con $\kappa_2(D) = 16$.

c) La Hessiana es D. El paso de Newton es $-D^{-1}Dx = -x$, por lo que el paso unitario lleva a $x = 0$. El modelo cuadrático de Taylor es exacto en todo punto y D es definida positiva.

14. Gradiente clásico y acelerado en Python

d) La misma función con momento 0 reproduce el gradiente clásico. Con momento 0.6 evalúa el gradiente en el punto extrapolado. El umbral común es $10^{-4} \cdot 8.5 = 0.00085$.

```
import numpy as np

diagonal = np.array([1.0, 16.0])
paso = 1 / 20

def objetivo(x):
    return (diagonal * x) @ x / 2

def historial(momento):
    x = np.ones(2)
    previo = x.copy()
    valores = [objetivo(x)]
    umbral = 1e-4 * valores[0]
    primera = None
    for k in range(1, 501):
        z = x + momento * (x - previo)
        previo, x = x, z - paso * diagonal * z
        valores.append(objetivo(x))
        if primera is None and valores[-1] <= umbral:
            primera = k
        if k >= 50 and primera is not None:
            return valores, primera
    raise RuntimeError("No se alcanzo el umbral en 500 pasos")

for nombre, momento in (("Gradiente", 0), ("Nesterov", 0.6)):
    valores, primera = historial(momento)
    print(nombre, [f"{valores[k]:.9g}" for k in (0, 10, 25, 50)])
    print("Primera iteracion:", primera)
```

k	Gradiente clásico	Nesterov
0	8.5	8.5
10	0.179242961	0.042494265
25	0.038472488	0.000315741
50	0.002960265	7.926410×10^{-8}
Primera iteración bajo el umbral	63	23

Optimización sin restricciones (continuación)

15. Búsqueda lineal y retroceso de Armijo

a) La dirección es $d = (-1, -16)^T$. Sea $\varphi(\alpha) = f(x^{(0)} + \alpha d)$. Al expandir,

$$\varphi(\alpha) = \frac{17}{2} - 257\alpha + \frac{4097}{2}\alpha^2.$$

Como $\varphi'' = 4097 > 0$, el cero de φ' da el mínimo sobre $\alpha \geq 0$ en $\alpha = 257/4097 \approx 0.06272883$.

b) Armijo exige $\varphi(\alpha) \leq 8.5 - \frac{257}{4}\alpha$.

α	$\varphi(\alpha)$	Cota de Armijo	Decisión
1	1800	-55.75	Rechaza
1/2	392.125	-23.625	Rechaza
1/4	72.28125	-7.5625	Rechaza
1/8	8.3828125	0.46875	Rechaza
1/16	0.439453125	4.484375	Acepta

c) Se tiene $\varphi'(1/16) = -257 + 4097/16 = -15/16$ y $c_2\varphi'(0) = -231.3$. Puesto que $-15/16 \geq -231.3$, el paso aceptado también cumple la condición de curvatura de Wolfe.

16. Una actualización BFGS

a) Como $y^T s > 0$, existe $\rho = (y^T s)^{-1}$. Se tiene $(I - \rho y s^T)y = y(1 - \rho s^T y) = 0$. Al multiplicar la fórmula de BFGS por y, queda $H_1 y = \rho s(s^T y) = s$.

b) Aquí $\nabla f(x_1, x_2) = (x_1, 4x_2)^T$. Con $H_0 = I$, $d_0 = (-2, -4)^T$ y $x^{(1)} = (3/2, 0)^T$. Por tanto, $s = (-1/2, -1)^T$, $y = (-1/2, -4)^T$, $y^T s = 17/4 > 0$ y $\rho = 4/17$.

c) El factor izquierdo en la actualización es $I - \rho s y^T = \frac{1}{17}\begin{pmatrix} 16 & -8 \\ -2 & 1 \end{pmatrix}$. Como $H_0 = I$, se multiplica por su traspuesta y se suma $\rho s s^T$. Se obtiene

$$H_1 = \frac{1}{289}\begin{pmatrix} 337 & -6 \\ -6 & 73 \end{pmatrix}, \qquad d_1 = -H_1 \nabla f(x^{(1)}) = \frac{1}{578}\begin{pmatrix} -1011 \\ 18 \end{pmatrix}.$$

La derivada direccional es $\nabla f(x^{(1)})^T d_1 = -3033/1156 < 0$, así que d_1 es de descenso.

Programación lineal

17. El problema dual y la dualidad débil

a) Sean x factible para el primal y $y \geq 0$ tal que $A^T y \geq c$. La no negatividad de x permite multiplicar cada desigualdad $(A^T y)_j \geq c_j$ por x_j y sumar. De modo análogo, la no negatividad de y permite multiplicar cada desigualdad $(Ax)_i \leq b_i$ por y_i y sumar. Resulta

$$c^T x \leq x^T A^T y = y^T A x \leq y^T b = b^T y.$$

Esta es la *dualidad débil*. Cada y que satisface las dos condiciones da una cota superior para todos los valores del primal. El *dual* busca la menor cota de esta forma,

$$\text{mín}\{b^T y : A^T y \geq c,\ y \geq 0\}.$$

Por cada fila de A hay una variable dual y por cada columna de A, una desigualdad en $A^T y \geq c$.

b) Al sumar $y^T(b - Ax)$ y $x^T(A^T y - c)$, los términos $-y^T Ax$ y $x^T A^T y$ se cancelan, pues son el mismo escalar con signos opuestos. En el primer producto, $y \geq 0$ y $b - Ax \geq 0$. En el segundo, $x \geq 0$ y $A^T y - c \geq 0$. Por eso cada producto es no negativo. La diferencia $b^T y - c^T x$ se llama *brecha de dualidad*. Si vale cero, para cualquier $\widetilde{x}$ primal factible y cualquier $\widetilde{y}$ dual factible, el inciso a) da

$$c^T \widetilde{x} \leq b^T y = c^T x \leq b^T \widetilde{y}.$$

Así, ningún primal factible mejora a x y ningún dual factible da una cota menor que y. Ambos son óptimos.

c) En las variables (x_1, x_2, t), las filas de A corresponden al presupuesto y a las restricciones $t - x_1 \leq 0$ y $2t - x_2 \leq 0$. Con multiplicadores $\lambda, \mu_1, \mu_2 \geq 0$, el dual es

$$\text{mín}\, M\lambda \qquad \text{sujeto a} \qquad p_1\lambda \geq \mu_1, \quad p_2\lambda \geq \mu_2, \quad \mu_1 + 2\mu_2 \geq 1.$$

Sea $k = p_1 + 2p_2$. Alcanzar t requiere $x_1 \geq t$ y $x_2 \geq 2t$, de modo que $kt \leq M$. Se alcanza $t^* = M/k$ con el presupuesto agotado en $x^W(p, M) = (M/k, 2M/k)$. Los precios positivos implican que cualquier aumento en una de esas dos cantidades excede el presupuesto, así que la canasta es única.

En el dual, $1 \leq \mu_1 + 2\mu_2 \leq k\lambda$. El vector $(\lambda, \mu_1, \mu_2) = (1/k, p_1/k, p_2/k)$ es factible y da el valor $M/k = t^*$. La brecha es cero y $\lambda^* = 1/k$.

d) **Canasta y gasto.** Alcanzar al menos $\bar{u}$ significa $x_1 \geq \bar{u}$ y $x_2 \geq 2\bar{u}$. Con precios positivos, la canasta de menor costo elige ambas cantidades mínimas. Por tanto,

$$h(p, \bar{u}) = (\bar{u}, 2\bar{u}), \qquad e(p, \bar{u}) = k\bar{u}, \qquad k = p_1 + 2p_2.$$

Misma canasta. Si $M = e(p, \bar{u}) = k\bar{u}$, la utilidad máxima del inciso c) es $M/k = \bar{u}$. Alcanzar una utilidad mayor requeriría gastar más que M. El hogar agota su ingreso en la canasta $(\bar{u}, 2\bar{u})$, por lo que $x^W(p, e(p, \bar{u})) = h(p, \bar{u})$.

Dos cambios y sus unidades. Aumentar en una unidad la meta de utilidad eleva el gasto mínimo en $k = e(p, 1)$ unidades monetarias. Aumentar en una unidad monetaria el ingreso eleva la utilidad máxima de M/k a $(M + 1)/k$, un incremento de $1/k = \lambda^*$. En este modelo el gasto es lineal en la meta. Por eso el costo monetario por unidad de utilidad y la utilidad por unidad monetaria son recíprocos, $\lambda^* = 1/e(p, 1)$.

Programación lineal (continuación)

18. Óptimos múltiples y región no acotada

a) La primera restricción impone $2x_1 + x_2 \leq 8$. En la recta de igualdad, $x_2 = 8 - 2x_1$. La segunda restricción equivale a $16 - 3x_1 \leq 10$, o $x_1 \geq 2$. La no negatividad de x_2 impone $x_1 \leq 4$. Por tanto, todos los óptimos tienen valor 8 y forman el segmento $\{(t, 8-2t) : 2 \leq t \leq 4\}$.

b) Para maximizar $x_1 - x_2$, la cota 2 se alcanza exactamente en los puntos $(2+t, t)$ con $t \geq 0$. Para maximizar $x_1 + x_2$, la dirección $d = (1, 1)$ desde $(0, 0)$ genera puntos factibles (t, t) para todo $t \geq 0$, y el objetivo $2t$ crece sin límite.

19. Regresión por desviaciones absolutas

a) Escriba $r_i = y_i - a - bx_i = p_i - q_i$ con $p_i, q_i \geq 0$. El programa lineal es

$$\text{mín} \sum_{i=1}^{5}(p_i + q_i), \qquad a + bx_i + p_i - q_i = y_i \quad (i = 1, \ldots, 5).$$

Los coeficientes a, b son libres. Para cualquier residuo, $p_i + q_i \geq |p_i - q_i| = |r_i|$ y la igualdad se logra con $p_i = \text{máx}(r_i, 0)$ y $q_i = \text{máx}(-r_i, 0)$. Eso prueba la equivalencia con minimizar la suma de residuos absolutos.

b)–c) La tabla muestra los dos ajustes antes y después de elevar el dato central. El código de la página siguiente resuelve ambos problemas y evalúa cada ajuste con las dos medidas de error.

y en $x = 2$	Ajuste	a	b	$\sum_i \lvert r_i \rvert$	$\sum_i r_i^2$
10	LAD	1	2	5	25
10	Mínimos cuadrados	2	2	8	20
20	LAD	1	2	15	225
20	Mínimos cuadrados	4	2	24	180

En mínimos cuadrados, $\bar{x} = 2$ y la pendiente es $\widehat{b} = \sum_i (x_i - \bar{x})y_i / \sum_i (x_i - \bar{x})^2$. El dato central aporta cero al numerador. Al aumentarlo en 10, $\bar{y}$ sube en 2 y también el intercepto $\widehat{a} = \bar{y} - \widehat{b}\bar{x}$.

Para verificar LAD analíticamente, denote por $\delta = a + 2b - 5$ el cambio en el valor ajustado central con respecto a la recta $1 + 2x$, y por $v = b - 2$ el cambio en pendiente. Sea $d = y_3 - 5 > 0$. Los cuatro datos restantes forman dos pares simétricos respecto de $x = 2$. Su contribución absoluta es $|\delta - v| + |\delta + v| + |\delta - 2v| + |\delta + 2v| \geq 4|\delta|$. Sumando el residuo central $|d - \delta| \geq d - |\delta|$, la pérdida es al menos $d + 3|\delta|$. La recta original alcanza d. Todo óptimo debe tener $\delta = 0$ y, en ese caso, la pérdida es $d + 6|v|$, por lo que también $v = 0$.

19. Código de los ajustes LAD y mínimos cuadrados

```
import numpy as np
from scipy.optimize import linprog

x = np.arange(5, dtype=float)
A = np.column_stack((np.ones_like(x), x))
I = np.eye(x.size)
Aeq = np.column_stack((A, I, -I))
objetivo = np.r_[0, 0, np.ones(2 * x.size)]
cotas = [(None, None)] * 2 + [(0, None)] * (2 * x.size)

for dato_central in (10, 20):
    y = np.array([1, 3, dato_central, 7, 9], dtype=float)
    res = linprog(objetivo, A_eq=Aeq, b_eq=y,
                  bounds=cotas, method="highs")
    if not res.success:
        raise RuntimeError(res.message)
    mc = np.linalg.lstsq(A, y, rcond=None)[0]
    for nombre, coef in (("LAD", res.x[:2]), ("MC", mc)):
        r = y - A @ coef
        print(dato_central, nombre, coef,
              np.abs(r).sum(), r @ r)
```